

WKUCourseKit Final Project Report

CPS 3320 Python Programming - Group Project

1. Problem Description

Students often need to check course lists, syllabus information, material requirements, ISBN records, and printable course packets separately. This can make course preparation inefficient and easy to forget. WKUCourseKit solves this problem with a local Python web application that organizes enrolled courses, syllabus metadata, required and optional materials, and print-friendly course packets in one place.

2. Methodology

The project runs entirely on the local machine. First, the user installs the Python dependencies from requirements.txt. Next, scripts/seed_db.py reads the included JSON dataset at data/mock_syllabus.json and writes normalized records into a local SQLite database. Finally, uvicorn starts the FastAPI application, and the user opens the local website in a browser.

The application uses SQLAlchemy model classes to represent terms, instructors, courses, enrollments, syllabi, syllabus sections, materials, course-material links, and student material statuses. Python service functions handle data import, search, filtering, material grouping, ISBN handling, and page context preparation. Jinja2 templates turn the processed data into browser pages, while CSS provides readable screen and print layouts.

The included dataset makes the demo repeatable, while an optional Simple Syllabus browser import/scraping flow can be used when a student signs in directly through Kean. That optional flow imports authorized syllabus records into the same local SQLite database and does not store Kean passwords.

Class concepts applied:

- Input, processing, and output: JSON data is loaded, processed, and displayed as local web pages.
- Decision structures and loops: filters use conditional logic, and the seed process loops through terms, courses, sections, and materials.
- Functions and modular design: the program is separated into small reusable service functions.
- Files and exceptions: the application reads local data files and handles possible file or sync errors.
- Lists, dictionaries, strings, and sets: these structures organize rows, counts, search text, translations, and duplicate handling.
- Classes and database programming: SQLAlchemy classes map Python objects to SQLite relational tables.

3. Tools and Libraries Used

Tool / Library	Purpose
Python 3.x	Primary programming language
FastAPI	Web framework and route handling
Jinja2	Server-rendered HTML templates
SQLite	Local relational database
SQLAlchemy	Database models, relationships, and queries

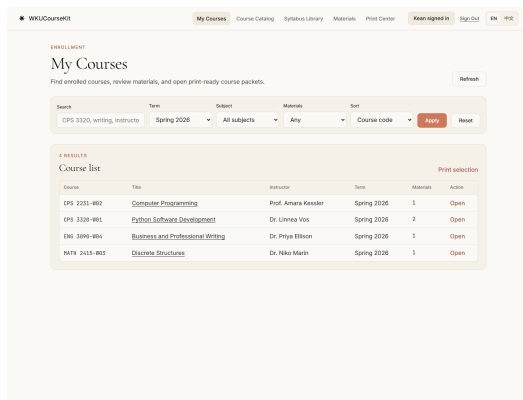
Pydantic	Structured data validation
Uvicorn	Local ASGI server used to run the FastAPI app
python-multipart	Form handling for local POST actions
Playwright	Optional local browser-assisted Simple Syllabus import

4. Results and Output Samples

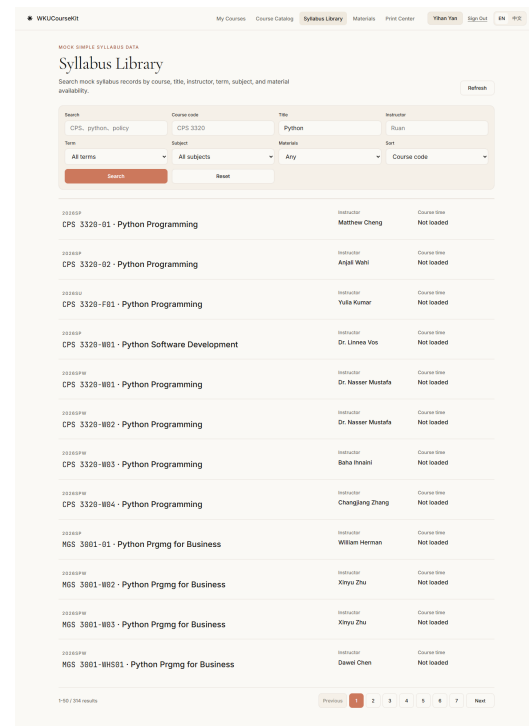
The finished application runs locally and displays the seeded dataset through multiple browser pages. The My Courses page lists enrolled courses with search and filters. The Syllabus Library page searches all course records. The Course Detail page presents metadata, instructor information, syllabus sections, and materials. The Materials page lists required and optional materials with ISBN values and source links. The Print Center creates print-friendly syllabus and material checklist output.

Output Sample	Result
python scripts/seed_db.py	2 terms, 8 instructors, 8 courses, 8 syllabi, 21 sections, 7 materials
/health	{"status":"ok","database":"connected"}
/courses	Displays enrolled course list with search, term filter, subject filter, and print action
/library	Displays searchable syllabus library with course, title, instructor, term, and subject filters
/courses/1	Displays course metadata, instructor information, syllabus sections, and materials
/materials	Displays required/optional materials with ISBN and source links
/print	Displays print-friendly syllabus packet and material checklist

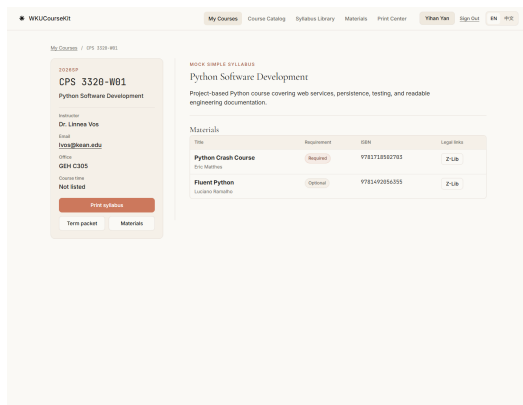
Screenshots included in output/results:



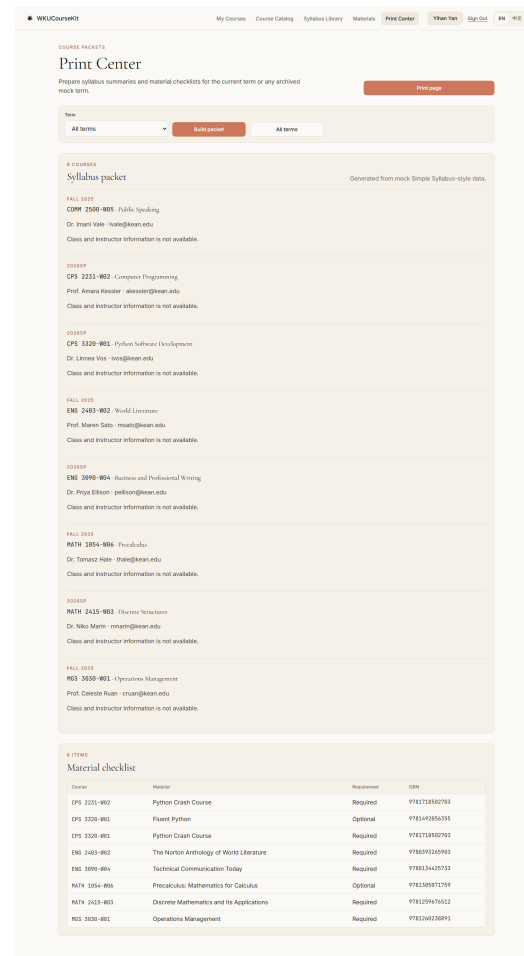
My Courses: enrolled course list with filters



Syllabus Library: searchable course records



Course Detail: syllabus reader and materials



Print Center: printable course packet

5. Challenges

The main challenge was keeping the project Python-first while still providing a useful web interface. The solution uses server-side GET filters, Python service functions, and Jinja2 templates instead of a heavy frontend framework. Another challenge was representing syllabus data in a normalized database. The schema separates courses, syllabi, syllabus sections, materials, and student statuses so records can be reused and searched efficiently. The optional Simple Syllabus import also needed a clear boundary: students sign in through Kean, and the local app imports only authorized syllabus data without storing passwords.

6. Conclusion

WKUCourseKit meets the final project requirement by providing a runnable local Python application with an included dataset, organized code, a clear README, visible output samples, a PDF report, and presentation slides. The project demonstrates practical use of Python functions, loops, conditionals, file input, data structures, classes, SQLite database programming, and local web output.